Google Developer Groups
on Campus | CUI Wah

COMSATS UNIVERSITY ISLAMABAD — WAH CAMPUS

3-DAY BOOTCAMP / CAPSTONE ASSIGNMENT

BEYOND PROMPTING.

Build an E-Commerce Website with a Mini CRM Dashboard.

DAY 01 · PROMPTING | DAY 02 · CONTEXT | DAY 03 · BUILDING

PROJECT BRIEF 01 / NEXUS-COMMERCE

From carefully designed prompts to a working, verifiable product.

Create a functional storefront, connect it to a simple customer and order management dashboard, document the actual prompts behind your features, and attempt deployment.

FINAL SUBMISSION DEADLINE

25 NOVEMBER 2026

Wednesday · 4:30 PM PKT (UTC+05:00)

Google Form shared on 20 November 2026. Link to follow from organizers.

Beyond Prompting Bootcamp — Capstone Assignment 2026

Project: NEXUS-COMMERCE — Prompt-Engineered E-Commerce Website + Mini CRM
Community: Google Developer Groups on Campus (GDGoC), CUI Wah
Bootcamp: 23–25 October 2026 | Prompt Engineering • Context Engineering • Application Building
Submission form publication: 20 November 2026 (link to be shared by organizers)
Final deadline: 25 November 2026, 4:30 PM Pakistan Standard Time (PKT / UTC+05:00)

1. Mission

Build a demonstrable e-commerce website **with a small customer relationship management (CRM) interface** using prompt engineering and context engineering techniques from the bootcamp. Your website should solve a coherent business problem, not just display an attractive static homepage. Document exactly how your prompts and context led to implementation decisions.

2. Project problem statement

Small online sellers often struggle to keep product listings, customer inquiries, cart activity, and orders in one consistent workflow. A buyer needs a clear product discovery and checkout journey, while the seller needs a way to view customers, track order progress, and respond to inquiries. Design **NEXUS-COMMERCE**, a practical, testable website prototype connecting a customer storefront with a merchant CRM dashboard. Use AI assistance responsibly during development, improve and verify your prompts, and make the work reproducible from GitHub.

The accompanying `docs/ENTERPRISE_REFERENCE.md` preserves the ambitious enterprise-grade NEXUS-COMMERCE vision supplied by the organizers. **That 21–28-day system is an advanced reference, not the minimum completion standard for this student capstone.**

3. Minimum working product (required)

A. Customer storefront

- Responsive homepage and product catalog with product image, name, price, and availability indicator.
- Search or category filtering, plus an individual product-detail view.
- Add/remove/update quantities in a cart with a visible total.
- A **simulated checkout** producing an order confirmation and an order visible to the merchant view. **Do not collect real card numbers or process live payments.**

B. Merchant CRM / admin dashboard

- Clear customer or inquiry list using **fictional demo records only**.
- Order list with order ID, item details, customer reference, total, and status.
- Basic order-status update or customer-inquiry status / notes feature.
- At least one useful summary metric, e.g., total demo orders or open inquiries.

C. Quality, context and verification

- Connect the two interfaces to coherent sample data and preserve changes during the demo (via suitable local persistence or a database). Explain the persistence scope honestly.
- Handle empty states, invalid form inputs, missing products and checkout errors.
- Demonstrate reasonable mobile and desktop layouts and navigation.
- Keep all passwords, API keys and secrets out of GitHub; use `.env.example` for placeholders and a `.gitignore`.
- Test the core flow: browse → cart → simulated checkout → view/manage the order in CRM.

D. AI learning requirement

AI-assisted creation with recorded prompts is required. A live AI chatbot in the finished website is optional. Show at least one clear example of applying Day 2 context engineering: e.g., a product JSON dataset, customer support policy, or explicit design/technical constraints supplied to the AI, followed by validation against actual application data.

4. Prompt engineering evidence (required)

Create a `prompts/` directory. **Every major section of the website that you built with AI assistance must have its own file** describing the issue, the prompt(s) and the implemented result. For instance: `01-storefront.md`, `02-product-search.md`, `03-cart-checkout.md`, `04-crm-dashboard.md`, `05-bug-fixes.md`. These are illustrative names; add or rename files to match your real project. Include an `INDEX.md` that links sections to source files.

For each documented issue or feature, record:

1. **Feature/problem:** what needed to be built or fixed and why.
2. **Prompt exactly as used:** your own actual request to the AI assistant (redact private details).
3. **Context supplied:** data, acceptance criteria, relevant file names and constraints.
4. **Iteration:** meaningful revised prompts and how you improved them.
5. **Implementation mapping:** source paths, components or commits that resulted.
6. **Verification:** what you personally tested and the observed result, including any remaining limitation.

The example in `templates/PROMPT_LOG_TEMPLATE.md` is an **illustrative blank template**, not an acceptable substitute for your actual prompts. Do not fabricate prompts, tests, screenshots or AI outputs.

5. GitHub repository requirements

Create **your own GitHub repository** for the solution (do not submit only a link to the organizer's learning repository). Keep it readable and accessible to evaluators. A recommended minimal layout is:

```
nexus-commerce-yourname/
├── README.md                          # Overview, features, setup and screenshots
├── .gitignore
├── .env.example                       # Placeholders only; never real secrets
├── prompts/
│   ├── INDEX.md                       # Map each major section to its prompt log
│   ├── 01-storefront.md
│   ├── 02-cart-checkout.md
│   ├── 03-crm-dashboard.md
│   └── 04-debugging-and-iterations.md
├── docs/
```

```
│   ├── architecture.md              # Short explanation or diagram
│   ├── testing.md                   # Real manual tests and outcomes
│   └── deployment.md                # Live link OR honest attempt log
├── screenshots/                    # Proof of actual working screens
├── frontend/                       # React/Next.js or other chosen frontend
├── backend/                        # If using a separate API
└── ...                              # Dependencies/config for your stack
```

This is a **suggested structure, not a requirement to create empty folders or pretend components exist**. A full enterprise monorepo with microservices, Kafka, Kubernetes, vector databases and AI agents is optional extension work.

6. Recommended development plan

Phase	Suggested dates	Output
Plan & prompts	10–11 Nov	Scope, user stories, rough wireframes, first prompt records
Build storefront	12–15 Nov	Product catalog, details, search/filter, cart
Build CRM & orders	16–19 Nov	Simulated checkout, order records, CRM list/status
Polish & deploy	20–22 Nov	Responsive fixes, deployment attempt, screenshots
Verify & submit	23–25 Nov	Test report, README, prompt logs, final form submission

Google Form: Organizers will share it on **20 November 2026**. Submit no later than **25 November 2026 at 4:30 PM PKT (UTC+05:00)**. This document does not provide or imply an existing form link.

7. Deployment

Make a genuine deployment attempt using a compatible platform such as **Vercel or Netlify** (supported frontend deployments), **Render** (backend services), or **Streamlit Community Cloud** (Streamlit-based prototype). You may use other suitable platforms.

- If deployed successfully, include the public URL in both the GitHub README and submission form.
- If not deployed successfully, include a clear deployment-attempt log with platform, steps taken, error/limitation, and local run instructions. A broken or fabricated link does not count as a deployment.
- **Docker is optional.** Do not add Kubernetes or Docker files solely for appearances.

8. Final submission checklist

- [ ] GitHub repository link to **my own** solution, accessible for review.
- [ ] README with project problem, features, chosen stack, prerequisites and exact setup commands.
- [ ] Working storefront with product discovery, cart and simulated checkout.
- [ ] Working demo CRM with customer/inquiry and order management.
- [ ] `prompts/` documentation for each major AI-built section, real prompts and iterations.
- [ ] At least one recorded context engineering example and verification note.
- [ ] Screenshots or other demonstrable evidence; no fake results.
- [ ] Deployment URL if live, or deployment-attempt log if unsuccessful.

- [ ] `.env.example`, `.gitignore`, and no credentials/real payment information.
- [ ] Submit the Google Form once it is released on 20 November, **before 4:30 PM PKT on 25 November 2026**.

9. Recommended evaluation rubric (proposed; organizer to confirm)

Criterion	Suggested points
Functional storefront, search, cart and simulated checkout	25
Useful CRM/admin flows and coherent data	20
Actual prompt logs, iterations and feature-level traceability	25
Context engineering, safety and real verification	10
GitHub README, organization and reproducibility	10
Deployment attempt, evidence and user experience	10
Total	**100**

This rubric is a **proposal for organizers**, not a confirmed certificate or grading policy. Certificates, if any, follow the organizers' separately announced requirements.

10. Optional advanced NEXUS-COMMERCE challenges

Pick only if your MVP is stable: authenticated role separation; a grounded AI shopping assistant; hybrid RAG over actual catalog data; image search; authorized order support tools with human approval; safe dynamic discounts; concurrency and idempotency protection; database-backed inventory; CI, Docker, observability, microservices, Kafka and load testing. The original 7-service architecture, agent system, SLA targets and chaos tests are in `docs/ENTERPRISE_REFERENCE.md`.

11. Learning references

- Day 1 + Day 2 source repository: https://github.com/UbaidGhazi/Beyond-prompting
- The source README describes an interactive **2-day** prompt/context engineering lab. This capstone corresponds to the separate **Day 3 application-building** bootcamp announcement shared by organizers.

12. Example of a specific prompt (illustration only)

A vague prompt like *"make my store"* leaves too much undefined. A stronger educational example is:

> **Example only — do not claim it as your real prompt:** "Build a responsive React product grid using `products.json` with name/category search and an empty-results state. Do not invent products or add live payments. Include acceptance criteria and a test checklist."

Use your **own actual prompts** in the submission. Record the context you supplied, how you refined the prompt, which files changed, and what you personally tested.

Learn • Build • Grow — GDGoC CUI Wah

Appendix A — Enterprise NEXUS-COMMERCE architecture (optional)

The **original technical vision** supplied with this assignment describes a **21–28-day** enterprise implementation. It is not required for the student MVP. These are ambitious extension targets for students with sufficient time and engineering experience.

Capability area	Original enterprise reference objectives
Distributed transactions	Seven decoupled microservices; gRPC/Kafka or RabbitMQ; inventory reservation and payment Saga rollback; idempotency and locking
Identity and catalog	OAuth/OIDC, signed JWT, multi-tenant RBAC; catalog variants; multimodal embedding indexing and CDC
Conversational intelligence	Supervisor agent, hybrid BM25/vector RAG, image similarity, reranking, authenticated fulfillment tools, HITL controls, grounded answers
Business intelligence	Dynamic pricing within margin rules; fraud/anomaly scoring; Kafka stream analytics
Merchant operations	GMV/conversion dashboard, agent traceability, support intervention, policy management
Cloud engineering	Docker, Kubernetes/Helm, GitHub Actions CI, Trivy, Prometheus, Grafana, OpenTelemetry, central logs
Verification	C4/ERD/sequence diagrams, load testing, fault injection and observability reports

Original suggested performance and stress targets

- Core API: **P99 < 150 ms**; hybrid search: **P95 < 250 ms**.
- Streaming AI first-token latency: **< 400 ms**; multistep tool execution: **P95 < 1,200 ms**.
- Stress scenario: **2,000** simultaneous checkout clients targeting a SKU with **50 items**; reference target of exactly 50 successful reservations, no negative inventory and no unhandled HTTP 5xx errors.
- Risk engine target: checkout scoring in **< 50 ms**, plus a chaos test that removes the vector database or external LLM.

Do not report these numbers as achieved unless you ran reproducible measurements. They are targets described in the original brief, not verified capabilities of a student project.

Enterprise repository map

The organizer-supplied tree groups code under `apps/`, `services/`, `ai-engine/`, `analytics-pricing/`, `shared/`, `build/`, `observability/`, `proto/`, `docs/` and `tests/`, alongside GitHub Actions and root configuration files.

The **complete** original technical problem statement and reference architecture tree are retained in the downloadable repository pack at `docs/ENTERPRISE_REFERENCE.md`. For this student assignment, **use only the folders you actually need**.